Guida Avvio VoiceScribe AI

Guida Avvio VoiceScribe AI — Server Ubuntu 22.04

Requisiti Hardware

Componente	Minimo
RAM	**40 GB** (32 GB riservati al ramdisk)
GPU	NVIDIA con VRAM sufficiente (RTX 3090/4090 consigliata)
Storage	50+ GB disponibili
OS	Ubuntu 22.04 LTS

FASE 1 — Preparazione Sistema

1.1 Aggiornamento sistema e dipendenze base

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y \
  ca-certificates curl gnupg lsb-release \
  ffmpeg make jq git openssl python3 python3-pip
```

1.2 Installazione Docker Engine

```
# Aggiungi chiave GPG e repository ufficiale Docker
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin

# Aggiungi utente corrente al gruppo docker (evita sudo)
sudo usermod -aG docker $USER
newgrp docker

# Verifica
docker --version
docker compose version
```

1.3 Installazione NVIDIA Container Toolkit (per GPU)

```
# Driver NVIDIA (se non già presenti)
sudo apt install -y nvidia-driver-535
# Riavvia dopo l'installazione del driver
# sudo reboot

# NVIDIA Container Toolkit
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
  | sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg

curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list \
  | sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg]
        https://#g' \
  | sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list

sudo apt update
sudo apt install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker

# Verifica GPU visibile a Docker
nvidia-smi
docker run --rm --gpus all nvidia/cuda:12.0-base-ubuntu22.04 nvidia-smi
```

1.4 Installazione Python e dipendenze Alembic (per migrazioni)

```
pip3 install --user alembic psycopg[binary] sqlalchemy
# Aggiungi al PATH se necessario
export PATH="$HOME/.local/bin:$PATH"
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
```

FASE 2 — Clonazione Repository

```
# Clona il monorepo nella home
git clone <URL_REPO> ~/voicescribe
cd ~/voicescribe

# Verifica struttura attesa
ls -la
# Dovresti vedere: voicescribe-infra/, voicescribe-api-gateway/, ecc.
```

FASE 3 — Configurazione Variabili d'Ambiente

```
cd ~/voicescribe/voicescribe-infra

# Copia il file di esempio
cp .env.example .env

# Modifica con valori reali
nano .env
```

Variabili obbligatorie da cambiare nel `.env`

```
# === DATABASE ===
POSTGRES_PASSWORD=<password_sicura_postgres>
VOICESCRIBE_DB_PASSWORD=<password_sicura_app>
```

```
DATABASE_URL=postgresql+psycopg://voicescribe_app:<password_sicura_app>@postgres:5432/voicescribe

# === REDIS ===
REDIS_PASSWORD=<password_sicura_redis>

# === SICUREZZA ===
INTERNAL_SERVICE_TOKEN=<token_interno_almeno_32_char>
JWT_SECRET_KEY=<chiave_jwt_almeno_32_char>

# === GRAFANA ===
GRAFANA_ADMIN_PASSWORD=<password_grafana>

# === HUGGINGFACE (solo per diarization PRO) ===
# HUGGINGFACE_TOKEN=hf_xxxxxxxxxxxxx

# === DOMINIO (opzionale in locale) ===
VOICESCRIBE_DOMAIN=voicescribe.local

# === RAMDISK ===
RAMDISK_PATH=/mnt/ramdisk
RAMDISK_SIZE=32G
```

Generare valori sicuri casuali

```
# Token interno
openssl rand -hex 32

# JWT secret
openssl rand -hex 32

# Password database
openssl rand -base64 24 | tr -d '=/+'
```

FASE 4 — Certificati TLS Self-Signed (sviluppo/test)

```
cd ~/voicescribe/voicescribe-infra

# Dai permessi agli script
make chmod-scripts

# Genera certificati self-signed per localhost (validi 365 giorni)
bash certs/generate-dev-certs.sh 365 localhost

# Verifica presenza certificati
ls -la certs/
# Atteso: voicescribe.crt, voicescribe.key
```

FASE 5 — Setup Ramdisk (32 GB tmpfs)

Richiede almeno 40 GB di RAM fisica

```
cd ~/voicescribe/voicescribe-infra

make ramdisk-setup
# Lo script: crea /mnt/ramdisk, aggiunge voce a /etc/fstab, monta tmpfs
```

```
# Verifica montaggio
df -h /mnt/ramdisk
# Atteso: filesystem tmpfs 32G
mountpoint /mnt/ramdisk
# Atteso: /mnt/ramdisk is a mountpoint
```

FASE 6 — Build e Avvio dello Stack

```
cd ~/voicescribe/voicescribe-infra

# Build immagini (prima volta: 10-20 min)
docker compose build

# Avvia tutti i container in background
make up
# equivalente a: docker compose up -d

# Verifica stato container
make status
# equivalente a: docker compose ps
```

Output atteso (`make status`)

```
NAME                                    STATUS
voicescribe-postgres                    Up (healthy)
voicescribe-redis                       Up (healthy)
voicescribe-nginx                       Up (healthy)
voicescribe-api-gateway                 Up
voicescribe-file-ingestion              Up
voicescribe-quota-manager               Up
voicescribe-audio-preprocessor          Up
voicescribe-audio-preprocessor-worker   Up
voicescribe-job-orchestrator            Up
voicescribe-transcription-engine        Up
voicescribe-diarization-engine          Up
voicescribe-export-service              Up
voicescribe-export-worker               Up
voicescribe-prometheus                  Up (healthy)
voicescribe-grafana                     Up (healthy)
```

FASE 7 — Migrazioni Database

```
cd ~/voicescribe/voicescribe-infra

# Carica variabili dal .env nella shell corrente
export $(grep -v '^#' .env | xargs)

# Esegui migrazioni Alembic
make migrate
# equivale a: alembic -c migrations/alembic.ini upgrade head

# Verifica con query diretta
docker compose exec postgres psql -U "$POSTGRES_USER" -d "$VOICESCRIBE_DB_NAME" \
  -c "\dt"
# Atteso: tabelle tenants, users, jobs, free_tier_usage, ecc.
```

FASE 8 — Seed Utenti di Test

```
cd ~/voicescribe/voicescribe-infra

# Inserisce utenti test (free@test.local / password)
make seed-e2e

# Verifica
docker compose exec postgres psql -U "$VOICESCRIBE_DB_USER" -d "$VOICESCRIBE_DB_NAME" \
  -c "SELECT email, tier FROM users WHERE email = 'free@test.local';"
```

FASE 9 — Healthcheck Completo

```
cd ~/voicescribe/voicescribe-infra

make health
```

Output atteso

```
SERVICE    STATUS   URL
-------    ------   ---
SVC-01     OK       http://localhost:8000/health
SVC-02     OK       http://localhost:8001/health
SVC-03     OK       http://localhost:8002/health
SVC-04     OK       http://localhost:8003/health
SVC-05     OK       http://localhost:8004/health
SVC-06     OK       http://localhost:8005/health
SVC-07     OK       http://localhost:8006/health
SVC-08     OK       http://localhost:8007/health
NGINX      OK       http://localhost:8080/health
[healthcheck] Tutti i servizi configurati rispondono
```

FASE 10 — Test Interno Manuale (cURL)

10.1 Test endpoint pubblico via Nginx

```
# Health globale tramite Nginx (HTTP dev sulla porta 8080)
curl -s http://localhost:8080/health | jq .

# HTTPS (self-signed, ignora verifica cert)
curl -sk https://localhost/health | jq .
```

10.2 Login Free Tier e ottenimento JWT

```
TOKEN=$(curl -s -X POST http://localhost:8080/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"free@test.local","password":"password"}' \
  | jq -r '.access_token')

echo "Token: $TOKEN"
```

10.3 Upload file audio per trascrizione

```
# Crea un file audio di test con ffmpeg (10 secondi, tono sinusoidale)
ffmpeg -f lavfi -i "sine=frequency=440:duration=10" -ar 16000 /tmp/test_audio.mp3

# Upload per trascrizione
JOB_RESPONSE=$(curl -s -X POST http://localhost:8080/v1/transcribe \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@/tmp/test_audio.mp3;type=audio/mpeg")

echo "$JOB_RESPONSE" | jq .
JOB_ID=$(echo "$JOB_RESPONSE" | jq -r '.job_id')
echo "Job ID: $JOB_ID"
```

10.4 Polling stato job fino a DONE

```
for i in $(seq 1 60); do
  STATUS=$(curl -s http://localhost:8080/v1/jobs/$JOB_ID \
    -H "Authorization: Bearer $TOKEN" | jq -r '.status')
  echo "[$i] Status: $STATUS"
  [ "$STATUS" = "DONE" ] && break
  [ "$STATUS" = "FAILED" ] && echo "Job FALLITO" && break
  sleep 5
done
```

10.5 Download trascrizione

```
# TXT (disponibile Free Tier)
curl -s http://localhost:8080/v1/jobs/$JOB_ID/download/txt \
  -H "Authorization: Bearer $TOKEN" \
  -o /tmp/trascrizione.txt
cat /tmp/trascrizione.txt

# SRT (disponibile Free Tier)
curl -s http://localhost:8080/v1/jobs/$JOB_ID/download/srt \
  -H "Authorization: Bearer $TOKEN" \
  -o /tmp/trascrizione.srt
cat /tmp/trascrizione.srt

# DOCX (solo PRO — deve restituire 403 per Free Tier)
curl -sv http://localhost:8080/v1/jobs/$JOB_ID/download/docx \
  -H "Authorization: Bearer $TOKEN" 2>&1 | grep "< HTTP"
# Atteso: HTTP/1.1 403
```

FASE 11 — Test E2E Automatici

```
cd ~/voicescribe/voicescribe-infra

# Installa dipendenze pytest
pip3 install --user pytest pytest-asyncio httpx

# Fornisci file audio di test (2 minuti)
cp /percorso/al/tuo/audio.mp3 tests/e2e/fixtures/test_audio_2min.mp3

# Esegui test scenario 1 (Free Tier full flow)
E2E_BASE_URL=http://localhost:8080 \
E2E_VERIFY_SSL=false \
pytest tests/e2e/test_scenario1_free_tier.py -v -m e2e
```

```
# Oppure tutti gli scenari e2e
make e2e
```

Comandi Operativi Utili

```
# Log di tutti i servizi (segui in real-time)
make logs

# Log servizio specifico
make logs-nginx
make logs-postgres
make logs-redis

# Stato GPU
nvidia-smi
watch -n 2 nvidia-smi

# Stato ramdisk
df -h /mnt/ramdisk

# Backup database
make backup

# Fermare lo stack
make down

# RESET COMPLETO (distruttivo — cancella tutti i volumi)
make reset CONFIRM=yes
```

Dashboard di Monitoraggio

Servizio	URL locale	Credenziali
Grafana	`http://localhost:3000`	`GRAFANA_ADMIN_USER` / `GRAFANA_ADMIN_PASSWORD` dal `.env`
Prometheus	`http://localhost:9090`	nessuna autenticazione
Nginx (dev)	`http://localhost:8080`	—

Troubleshooting Rapido

```
# Container non parte: controlla i log
docker compose logs <nome-container>

# PostgreSQL non sano
docker compose exec postgres pg_isready -U postgres

# Redis non sano
docker compose exec redis redis-cli -a "$REDIS_PASSWORD" ping

# GPU non visibile nei container
nvidia-smi                              # Driver OK sull'host?
docker run --rm --gpus all nvidia/cuda:12.0-base-ubuntu22.04 nvidia-smi

# Ramdisk non montato dopo reboot
```

```
sudo mount /mnt/ramdisk
df -h /mnt/ramdisk

# Migrazioni fallite (DB non raggiungibile)
docker compose exec postgres psql -U postgres -c "SELECT now();"
```

Nota: I servizi SVC-06 (Transcription Engine) e SVC-07 (Diarization Engine) richiedono GPU NVIDIA. Senza GPU questi container non elaboreranno job, ma il resto della pipeline (ingestion, quota, export) rimane testabile.